

Assignment 1 — Deliverable 2

Static Code Quality Analysis of *smoosh*

Course: Software Quality Engineering **Student:** Pir Ahmed Shah / 24P-3000

Instructor: Sara Rehmat **Repository:**

<https://github.com/j-mcnamara/smoosh> **Analysis branch:** static-analysis-assignment **Tool:** Pylint 4.0.9 (Python 3.12.3)

1. Project description and code statistics

Project name: smoosh — *Software Module Outline & Organization Summary Helper*

Purpose: `smoosh` is a command-line tool that scans a repository or directory and produces a compact, LLM-friendly plain-text summary of the codebase (file tree + concatenated source). It uses `click` for the CLI, `rich` for terminal output, `PyYAML` for configuration, `chardet` for text detection and `pyperclip` for clipboard output.

Statistic	Value
Python source files (src/smoosh/)	15
Python LOC in src/smoosh/	1,376
Test files	2 (tests/test_cli.py, tests/test_core.py)
Existing automated tests	Yes (Pytest)
License	MIT

Statistic	Value
Dependencies	Self-contained; no database, cloud service, GPU or paid API

The package is organised into three collaborating subsystems plus the CLI:

- `analyzer/` — walks the repository and builds the file/tree model.
- `composer/` — concatenates and formats the collected content.
- `utils/` — configuration, logging, path and file helpers.
- `cli.py` — the `click` entry point that wires everything together.

2. Baseline

A separate analysis branch was created and an explicit marker commit was made before any modification:

```
git switch -c static-analysis-assignment

git commit --allow-empty -m "Baseline before static analysis"

git tag baseline-before-static-analysis
```

The original version is preserved in the upstream history (commit 9ea6335, tag 0.1.4). Baseline evidence, including the full Pylint output, is committed in `reports/pylint_baseline.txt`.

Analysis scope. Pylint is run over the application package `src/smoosh`. The `tests/` directory is verification code and the bundled `src/smoosh/pyperclip.pyi` is a third-party API stub, so neither is part of the code-quality scope. This scope also makes the baseline exactly match the Deliverable 1 figure (8.98/10).

3. Initial static analysis

Command:

```
pylint src/smoosh
```

Metric	Baseline
	8
	.
	9
Pylint	8
src	
or	/
e	
	1
	0
Total	
issues	3
	2
Convention	
	1
(C)	

Metric	Baseline
--------	----------

Error (E)	4
Refactoring (R)	2
Warning (W)	2
	5

Files with the most issues:

File	Issues
src/smoosh/analyzer/repository.py	10
src/smoosh/composer/	5

File	Issues
concatenator.py	
src/smoosh/cli.py	5
src/smoosh/__init__.py	4
src/smoosh/utils/logger.py	3
src/smoosh/utils/config.py	2
	1
path_resolver.py, formatter.py, tree.py	each

The complete raw output is reproduced in **Appendix A** and stored in `reports/pylint_baseline.txt`.

4. Classification of findings

The findings were grouped into meaningful software-quality concerns rather than Pylint's own letter codes.

Quality concern	Pylint messages	Count	Why it matters
Error	broad-exception-	3	Catching

Quality concern	Pylint messages	Count	Why it matters
handling	caught (W0718)		Exception hides programming errors and makes failures hard to diagnose.
Coding convention / performance	logging-fstring-interpolation (W1203)	7	f-strings are formatted even when the log level is disabled, wasting work

Quality concern	Pylint messages	Count	Why it matters
Unused / dead code	<code>unused-import</code> (W0611), <code>unnecessary-pass</code> (W0107)	1 0	and obscuring log structure. Dead code increases reading cost and can mislead maintainers.
	<code>no-value-for-parameter</code> (E1120)	4	Reported as missing arguments; here a known decorator

Quality concern	Pylint messages		Count	Why it matters
				ator false positi ve. A functi on with 19 locals is doing too much and is hard to test.
Complexity		too-many-locals (R0914)	1	
Maintainability / coupling		no-else-return (R1705), import-outside-toplevel (C0415)	2	Unnec essary branc hing and hidde n impor ts reduc e clarit y and

Quality concern	Pylint messages	Count	Why it matters
Maintainability			structure.
Reliability		2	Local dependencies in file I/O is a portability and data-integrity risk.
Naming readability		1	Shadowing the module logger is a readability/behaviour trap.
Documentation		2	Tracked,

Quality concerns

Pylint messages

Conclusion

Why it matters

ent
atio
n /
deb
t

intent
ional
techni
cal
debt
for
the
unfini
shed
fold/s
moos
h
mode
s.

The 25 warnings are dominated by low-risk but real hygiene problems (logging style, unused imports, redundant pass). Only 4 are "errors", and all 4 are the same Click false positive. The single genuinely structural finding is the over-long analyze_repository().

5. Detailed investigation of selected findings

Twelve findings are analysed below, covering seven different quality concerns. For each: **(A)** tool finding, **(B)** original code, **(C)** independent explanation, **(D)** quality impact, and the **Part 6 verdict** (A = should fix, B = context dependent, C = acceptable / false positive).

Finding 1 — Broad exception in the logger

A. src/smoosh/utils/logger.py:48 — W0718 (broad-exception-caught)

B. Original code

```
try:

    file_handler = logging.FileHandler(log_file)

    file_handler.setFormatter(file_formatter)

    logger.addHandler(file_handler)

except Exception as e:

    logger.warning(f"Failed to create log file at {log_file}:
{e}")
```

C. Explanation. `logging.FileHandler` can realistically fail with `OSError` (permissions, missing directory, bad path). Catching `Exception` additionally swallows programming errors such as a `TypeError` from a malformed `log_file`, reporting them as "failed to create log file" and continuing.

D. Impact. Reliability and diagnosability: real defects are converted into a soft warning and lost. Narrowing to `OSError` makes the intent explicit.

Verdict: A — should be fixed.

Finding 2 — Broad exception when reading files

A. `src/smoosh/analyzer/repository.py:148` — W0718 (broad-exception-caught)

B. Original code

```
except Exception as e:
```

```
    logger.warning(f"Error reading file {file_info.path}: {e}")
```

```
    file_info.content = None
```

C. Explanation. Reading a file can fail with `OSError` (I/O, permissions) or `UnicodeDecodeError` (bad encoding). Catching everything also hides bugs in the surrounding code and silently drops content.

D. Impact. Reliability and maintainability: failures become invisible and the reason for missing content is unclear.

Verdict: A — should be fixed.

Finding 3 — f-string in logging

A. `src/smoosh/analyzer/repository.py:69` — W1203 (logging-fstring-interpolation) (7 occurrences in total: `repository.py:69,72,118,149`, `logger.py:49`, `concatenator.py:52`, `path_resolver.py:69`)

B. Original code

```
logger.info(f"Git repository root detected at {input_path}")
```

C. Explanation. The f-string is evaluated eagerly, before the logger checks whether the level is enabled. The logging API is designed to receive a format string plus arguments and perform the interpolation lazily.

D. Impact. Maintainability and a minor performance cost. It also separates log data from log message, which is important for structured logging.

Verdict: A — should be fixed (mechanical and safe).

Finding 4 — Unused import in config

A. `src/smoosh/utils/config.py:3` — W0611 (unused-import)

B. Original code

```
import os
```

C. **Explanation.** `os` is imported but never used; the only reference is `PathLike = Union[str, "os.PathLike[str]"]`, a string annotation that is never evaluated. The alias itself is also unused in this module.

D. **Impact.** Readability: the reader must check whether `os` matters.

Verdict: A — should be fixed.

Finding 5 — Unused import in tree

A. `src/smoosh/analyzer/tree.py:3` — W0611 (unused-import)

B. Original code

```
import os

...

PathLike = Union[str, "os.PathLike[str]"]
```

C. **Explanation.** As above, the import is only "used" inside a quoted

forward-reference, so at runtime the name is never touched. The correct choice is either to use `os.PathLike` directly (unquoted) or to drop the import.

D. Impact. Readability and a small amount of confusion about typing.

Verdict: A — should be fixed.

Finding 6 — Redundant `pass` statements

A. `src/smoosh/__init__.py:24,30,36,42` — W0107 (unnecessary-pass) (also `concatenator.py:14`, `formatter.py:14`, `repository.py:156`)

B. Original code

```
class SmooshError(Exception):

    """Base exception class for smoosh."""

    pass
```

C. Explanation. A class whose only body is a docstring already has a valid body; `pass` is redundant. The same applies to `CompositionError` and `FormattingError`.

D. Impact. Dead code / readability — small, but it adds noise across the exception hierarchy.

Verdict: A — should be fixed.

Finding 7 — Too many locals

A. `src/smoosh/analyzer/repository.py:40` — R0914 (too-many-locals, 19/15)

B. Original code (abridged)

```
def analyze_repository(path, config, force_cat=False):

    input_path = Path(str(path))

    git_root = find_git_root(input_path)

    is_git_root = bool(git_root and git_root == input_path)

    root_path = input_path

    if is_git_root and git_root:

        logger.info(...)

        root_path = git_root

    else:
```

```
logger.info(...)

try:

    gitignore_patterns = set()

    if config["gitignore"]["respect"] and not force_cat:

        gitignore_root = git_root if git_root else root_path

        gitignore_patterns =
get_gitignore_patterns(str(gitignore_root))

    gitignore_patterns.add(".git/")

    max_size_mb = ...

    files, total_size_mb, python_files_count = [], 0.0, 0

    for file_path in walk_repository(...):

        ...
```

C. Explanation. One function resolves the root, builds ignore patterns, applies size limits, collects file records and aggregates statistics. Each step needs several locals, pushing the count to 19. The function mixes orchestration with detail, so it is difficult to unit-test the pieces.

D. Impact. Testability and maintainability: the function is hard to read and any single change risks the others. Decomposing improves cohesion.

Verdict: A — should be fixed (decompose rather than raise the limit).

Finding 8 — Import inside the function

A. `src/smoosh/analyzer/repository.py:35` — C0415 (`import-outside-toplevel`)

B. Original code

```
def get_tree_representation(self) -> str:

    """Compose a tree-style representation of the repository
    structure."""

    from .tree import generate_tree

    return generate_tree(str(self.root), self.files)
```

C. Explanation. The local import suggests a circular-dependency workaround, but `tree.py` only depends on `custom_types.py`, so a module-

level import is safe. Hidden imports complicate reasoning about module initialisation.

D. Impact. Maintainability and clarity of the dependency graph.

Verdict: B — context dependent. It is a legitimate pattern when a cycle truly exists; here it does not, so it was moved to the top. If a cycle existed, keeping the local import would be the correct choice.

Finding 9 — Unnecessary `elif` after `return`

A. `src/smoosh/composer/concatenator.py:102` — R1705 (no-else-return)

B. Original code

```
if mode == "cat":

    return compose_cat_mode(repo_info)

elif mode == "fold":

    return compose_fold_mode(repo_info)

elif mode == "smoosh":

    return compose_smoosh_mode(repo_info)
```

```
else:
```

```
    raise CompositionError(f"Unknown composition mode: {mode}")
```

C. Explanation. Once each branch returns, the `elif/else` chain adds indentation without adding meaning. A dispatch table expresses the same intent more directly and makes the set of supported modes obvious.

D. Impact. Readability and modifiability; adding a mode should be a one-line change.

Verdict: A — should be fixed.

Finding 10 — Missing explicit encoding

A. `src/smoosh/utils/config.py:114` — W1514 (unspecified-encoding) (also `cli.py:87`)

B. Original code

```
with open(config_path) as f:
```

```
    user_config = yaml.safe_load(f)
```

```
...
```

```
    output_path.write_text(result)
```

C. Explanation. With no `encoding=`, Python uses the platform's preferred

encoding, which differs between Linux, macOS and Windows. A YAML file with non-ASCII characters can then fail differently per machine.

D. Impact. Reliability and portability; it can also cause data corruption on write. Explicit UTF-8 removes the ambiguity.

Verdict: A — should be fixed.

Finding 11 — Shadowing the module logger

A. `src/smoosh/utils/logger.py:26` — W0621 (redefined-outer-name)

B. Original code

```
def setup_logger(name="smoosh", ...):  
  
    logger = logging.getLogger(name)    # local  
  
    ...  
  
# module level  
  
logger = setup_logger()
```

C. Explanation. Inside `setup_logger` the local `logger` shadows the module-level `logger` defined later in the same file. It works, but a reader cannot tell at a glance which `logger` a call refers to.

D. Impact. Readability and maintainability; renaming the local variable

removes the ambiguity.

Verdict: B — context dependent. Renaming is low-cost and worthwhile, but shadowing here is not a defect.

Finding 12 — Click `no-value-for-parameter`

A. `src/smoosh/cli.py:106` — E1120 (`no-value-for-parameter`) ×4 (`target`, `mode`, `output`, `force_cat`)

B. Original code

```
@click.command()

@click.argument("target", ...)

@click.option("--mode", ...)

def main(target, mode, output, force_cat):

    ...

if __name__ == "__main__":

    main()
```

C. Explanation. `click` reads the decorated signature and injects the arguments at runtime, but Pylint cannot see through the decorators and reports a call with no arguments. This is a known false positive.

D. Impact. None on the software; it only inflates the error count. It should be suppressed narrowly with an explanatory comment rather than by disabling the whole check.

Verdict: C — acceptable / false positive.

Finding 13 — Tracked TODOs

A. `src/smoosh/composer/concatenator.py:147,163 — W0511 (fixme)`

B. Original code

```
# TODO: Implement fold mode composition

return compose_cat_mode(repo_info)
```

C. Explanation. The `fold` and `smoosh` compression modes are advertised by the CLI but currently fall back to `cat`. The TODOs correctly mark incomplete work. The real issue is behavioural (the modes are placeholders), not the comment itself.

D. Impact. Documentation / honesty of the interface. Removing the TODOs would hide the gap.

Verdict: C — acceptable exception (legitimate tracked debt), though the placeholder behaviour should be closed before release.

6. Manual code review (independent of Pylint)

These five problems were found by reading the code; the first three were

not reported by Pylint at all.

M1. Duplicate exception class hides failures (*not reported by Pylint*)

`src/smoosh/analyzer/repository.py` (old) defined its own class:

```
class AnalysisError(Exception):  
  
    """Raised when repository analysis fails."""  
  
    pass
```

while `src/smoosh/__init__.py` defines `AnalysisError(SmooshError)`, and `cli.py` catches the package one:

```
except (ConfigurationError, AnalysisError, GenerationError) as e:
```

Because the two classes are unrelated, the exception raised by `analyze_repository()` was **never caught by that handler**; it fell through to the generic `except Exception`, printing "An unexpected error occurred!" and losing the specific message.

- **Problem:** duplicate, inconsistent exception hierarchy.
- **Impact:** maintainability and a real loss of error information (reliability).
- **Recommendation:** delete the local class and import the package-level `AnalysisError`. (*Implemented.*)

M2. Duplicated and inconsistent default configuration (*not reported by Pylint*)

```
DEFAULT_CONFIG: ConfigDict = {

    "output": {"max_tokens": 5000, ...},

    ...

}

def load_config(config_dir):

    default_config = {

        "output": {"max_tokens": 10000, ...},

        ...

    }
```

The module defines `DEFAULT_CONFIG` but `load_config()` ignores it and rebuilds its own defaults with a *different* `max_tokens` (10000 vs 5000). Three helper functions (`_merge_output`, `_merge_thresholds`, `_merge_gitignore`) were also dead code.

- **Problem:** duplicated logic and a contradictory source of truth.
- **Impact:** maintainability; a future edit to one default silently has no effect.
- **Recommendation:** single `DEFAULT_CONFIG` used by `load_config()`, remove the dead helpers. *(Implemented.)*

M3. Inefficient repository walk *(not reported by Pylint)*

```
for path in root.rglob("*"):

    if should_ignore_path(path, root, ignore_patterns):

        continue

    ...

    if not is_text_file(path):

        continue

    yield path
```

`rglob("*")` descends into **every** subdirectory — including the `.git` object store, `.venv`, `node_modules`, `cache`s — and only then filters. For each surviving path it may read up to 1 KB to guess the encoding. On a real repository this is a large amount of wasted I/O.

- **Problem:** performance and unnecessary coupling to `chardet`.
- **Impact:** performance/scalability.
- **Recommendation:** prune ignored directories during traversal (e.g. use `os.walk` and mutate `dirnames` in place) so ignored trees are never entered. *Identified but deliberately left as a follow-up (see §11).*

M4. Dead parameter in the tree formatter

```
def format_tree(node, prefix="", is_last=True,
include_indicators=True):

    ...

    line += format_tree(child_node, child_prefix, i == children_count -
1,

                        include_indicators)
```

`include_indicators` is threaded through the recursion but never actually used to change the output. Because it is passed along, Pylint's `unused-argument` does not fire.

- **Problem:** misleading public API / function design.
- **Impact:** readability and modifiability — callers believe they can toggle indicators.
- **Recommendation:** either implement the indicator output or drop the parameter.

M5. `TreeNode` misuses `@dataclass`

```
@dataclass

class TreeNode:

    name: str

    is_dir: bool

    children: Dict[str, "TreeNode"]

    is_python: bool = False

    def __init__(self, name, is_dir=True):

        self.name = name

        ...
```

The `@dataclass` decorator generates an `__init__` that is then immediately overridden by a hand-written one, so the field declarations are partly decorative. In addition `is_python` is set but never read anywhere.

- **Problem:** class-design confusion and an unused attribute.
- **Impact:** maintainability; the class pretends to be a dataclass but is not.
- **Recommendation:** use a real dataclass with `field(default_factory=dict)` and remove `is_python`, or drop `@dataclass`.

7. Pylint configuration and justification

A project-specific `.pylintrc` was generated and adapted. Three (plus one additional) decisions are documented; the justifications matter more than the values themselves.

#	Setting	Default	Decision	Justification
1	ignore-patterns	^ \ . #	Changed to → ^ \ . #	src/ smo/ sh/ pyp/ rcL/ p.p. i is a han d- wri en stu for

#

Setting

Default

Decision

,
.
*
\
.
p
y
i
\$

Justification

the
this
d-
par
y
pyp
rcl
p
API
Lin
ng
pro
uce
29
fin
ngs
abo
ut
som
eon
else
s
inte
fac
and
dis
rte
the
sco
e.
The

#	Setting	Default	Decision	Justification
2	max-line-length	100	Retention	The project's PEP 8 style guide recommends a maximum line length of 79 characters, but the project's documentation states that the maximum line length should be 100 characters. The project's documentation also states that the maximum line length should be 100 characters, but the project's documentation also states that the maximum line length should be 100 characters.

#

Setting

Default

Decision

Justification

3

max-locals

1
5

Retained

me
ns
one
wic
h is
enf
rce
by
eve
y
too

ana
yze
rep
sit
ry(
exc
ede
this
Rel
xin
the
thr
sho
d
wo
ld
hid
a
gen
ine

#

Setting

Default

Decision

Justification

4

no-value-for-parameter

enabled

Suppressed information

con-
ple-
ty
pro-
lem
so
the
fun-
tion
wa-
dec-
mp-
sed
ins-
ad.

The
4
E11
0
rep-
rts
are
a
Clic
fals
pos
ive
Rat
er
tha

#

Setting

Default

Decision

none

Justification

disabling the checker
k
glo
ally
(wh
ch
wo
ld
hid
rea
def
cts)
a
sin
e
exp
ana
ory

pyl
nt:
dis
ble
wa
pla
ed
the
call

#

Setting

D
e
f
a
u
l
t

D
e
c
i
s
i
o
n

Just
fica
ion

site

8. Code improvements (before / after)

Five significant improvements were made, four of them structural. Each was committed separately to keep the history meaningful.

Improvement 1 — Unify the exception hierarchy (*structural*)

Before (`repository.py`): a private `AnalysisError(Exception)` shadowed the public one, so `cli.py` could not catch analysis failures. **After:** `from .. import AnalysisError`; duplicate class deleted. **Why better:** errors now propagate to the intended handler with their specific message. Fixes the M1 defect. Also removed redundant `pass` statements from the package's exception classes.

Improvement 2 — Decompose `analyze_repository()` (*structural*)

Before: a single function with 19 locals doing four jobs. **After:** `_resolve_root()`, `_collect_ignore_patterns()`, `_build_file_info()` and `_collect_files()`, with `analyze_repository()` as a short orchestrator. The per-file handler was narrowed from `Exception` to `(OSError, ValueError)`. **Why better:** each helper has one responsibility, is independently testable, and the complexity warning disappears instead of being silenced.

Improvement 3 — Consolidate configuration (*structural*)

Before: `DEFAULT_CONFIG` (5000) vs a rebuilt local default (10000), plus three unused `_merge_*` helpers. **After:** one `DEFAULT_CONFIG` used by `load_config()`; dead helpers removed; config read with an explicit `encoding="utf-8"`; exception narrowed to `(OSError, yaml.YAMLError)`. **Why better:** a single source of truth for defaults and less code to maintain (M2).

Improvement 4 — Dispatch table for composition modes

Before: an if/elif/else chain in `compose_content()`. **After:** a `composers` mapping looked up by mode. **Why better:** removes the no-else-return warning and makes adding a mode a one-line change.

Improvement 5 — Logging, imports and encodings cleanup

Before: 7 eager f-string log calls, unused `os` imports, a shadowed module logger, an unnecessary function-local import and a non-UTF-8 write. **After:** lazy %-style logging, imports cleaned, local logger renamed to `log`, `generate_tree` imported at module level, and `output_path.write_text(..., encoding="utf-8")`. **Why better:** lower log overhead, clearer module dependencies, portable I/O.

9. Verification (Part 10)

Behaviour was verified by running the existing tests plus a new regression suite added in commit `ded64af` (7 new tests in `tests/test_core.py`), which covers the public analyzer, tree, composer and configuration APIs and an end-to-end CLI run.

```
$ pytest -q
```

```
.....
```

```
[100%]
```

```
9 passed in 0.15s
```

Coverage of the modified package:

```
$ pytest --cov=smoosh -q
```

```
TOTAL    452 stmts    94 miss    104 branch    19 partial    76%
```

A manual end-to-end run was also performed against a scratch directory and produced a correct tree + concatenated output file. The test that specifically guards the M1 fix is `test_analysis_error_is_the_package_exception`.

10. Final static analysis and comparison

```
pylint src/smoosh          # after refactoring
```

Metric	Before	After	Change
Pylint score	89.8	99.6	+10.0
Total findings	32	20	-12
Conventions	1	0	-1

Metric	Before	After	Change
Ention (C)			1
Error (E)	4	0	−4
Refactor in g (R)	2	0	−2
Warnings (W)	2	2	−
	5		3

The two remaining findings are the intentional TODOs for the unimplemented `fold/smoosh` modes.

Answers:

- **Did the score improve?** Yes, from 8.98 to 9.96.
- **Which categories improved most?** Warnings (−23) and errors (−4). The warning reduction came from the logging, encoding and dead-code fixes; the errors were the Click false positive plus the

now-fixed structural issues.

- **Did any new warnings appear?** No. The two remaining findings are the same intentional TODOs present in the baseline.
- **Does the numerical improvement reflect the quality improvement?** Largely, but not perfectly. Most of the score came from mechanical fixes (logging style, `pass` removal, unused imports) that are easy and low-risk. The single most important change — the M1 exception-hierarchy bug — was worth far more than its contribution to the score, and the highest-risk manual finding (M3, the repository walk) is **not** reflected in the score at all because Pylint cannot measure it. A high score is therefore evidence of hygiene, not proof of good design.

11. Static analysis vs human review

1. **What Pylint detects well.** Mechanical, rule-based issues: unused imports, redundant `pass`, missing encodings, eager logging, unnecessary `elif`, shadowed names and overly long functions. These are objective and reliably actionable.
2. **What required human judgment.** The duplicate `AnalysisError` (M1) is syntactically clean — Pylint saw nothing wrong — yet it silently broke error handling. The inconsistent configuration defaults (M2), the dead parameter (M4), the misused `dataclass` (M5) and above all the performance problem (M3) were only visible when reading for *intent*, not syntax.
3. **False positive / not worth fixing.** The $4 \times E1120$ reports are a known Click false positive. The 2 `fixme` notes are legitimate tracked debt and should not be removed merely to raise the score.
4. **Can a high score hide poor design?** Yes. Before this work the project already scored 8.98 while containing a real error-handling defect (M1) and a performance problem (M3). The score measures style compliance, not correctness or architecture.
5. **Should static analysis replace human review?** Neither replacement nor occasional use — it should **complement** review. Tools cheaply and consistently remove mechanical defects so that human reviewers can spend their attention on design, correctness

and intent, which is where the expensive defects live.

12. Git history

The work is split into meaningful commits rather than one final commit:

ded64af Add regression tests for refactored modules

b02932d Clean up imports, lazy logging, encodings and composition
dispatch

7089e9a Consolidate configuration defaults and remove dead helpers

e8ded7a Decompose repository analysis into focused helpers

57110a5 Unify exception hierarchy and remove redundant pass statements

c738e64 Add project-specific Pylint configuration

bc26c2f Record baseline Pylint analysis before changes

c343dfc Baseline before static analysis

9ea6335 increment version (upstream baseline)

Appendix A — Complete initial Pylint output

```
***** Module smoosh
```

```
src/smoosh/__init__.py:24:4: W0107: Unnecessary pass statement  
(unnecessary-pass)
```

```
src/smoosh/__init__.py:30:4: W0107: Unnecessary pass statement  
(unnecessary-pass)
```

```
src/smoosh/__init__.py:36:4: W0107: Unnecessary pass statement  
(unnecessary-pass)
```

```
src/smoosh/__init__.py:42:4: W0107: Unnecessary pass statement  
(unnecessary-pass)
```

```
***** Module smoosh.cli
```

```
src/smoosh/cli.py:87:16: W1514: Using open without explicitly  
specifying an encoding (unspecified-encoding)
```

```
src/smoosh/cli.py:106:4: E1120: No value for argument 'target' in  
function call (no-value-for-parameter)
```

```
src/smoosh/cli.py:106:4: E1120: No value for argument 'mode' in  
function call (no-value-for-parameter)
```

```
src/smoosh/cli.py:106:4: E1120: No value for argument 'output' in  
function call (no-value-for-parameter)
```

```
src/smoosh/cli.py:106:4: E1120: No value for argument 'force_cat' in  
function call (no-value-for-parameter)
```

```
***** Module smoosh.utils.path_resolver
```

```
src/smoosh/utils/path_resolver.py:69:8: W1203: Use lazy % formatting in  
logging functions (logging-fstring-interpolation)
```

```
***** Module smoosh.utils.logger
```

```
src/smoosh/utils/logger.py:26:4: W0621: Redefining name 'logger' from  
outer scope (line 55) (redefined-outer-name)
```

```
src/smoosh/utils/logger.py:48:15: W0718: Catching too general exception  
Exception (broad-exception-caught)
```

```
src/smoosh/utils/logger.py:49:12: W1203: Use lazy % formatting in  
logging functions (logging-fstring-interpolation)
```

```
***** Module smoosh.utils.config
```

src/smoosh/utils/config.py:114:17: W1514: Using open without explicitly specifying an encoding (unspecified-encoding)

src/smoosh/utils/config.py:3:0: W0611: Unused import os (unused-import)

***** Module smoosh.composer.concatenator

src/smoosh/composer/concatenator.py:147:5: W0511: TODO: Implement fold mode composition (fixme)

src/smoosh/composer/concatenator.py:163:5: W0511: TODO: Implement smoosh mode composition (fixme)

src/smoosh/composer/concatenator.py:14:4: W0107: Unnecessary pass statement (unnecessary-pass)

src/smoosh/composer/concatenator.py:52:12: W1203: Use lazy % formatting in logging functions (logging-fstring-interpolation)

src/smoosh/composer/concatenator.py:102:4: R1705: Unnecessary "elif" after "return", remove the leading "el" from "elif" (no-else-return)

***** Module smoosh.composer.formatter

src/smoosh/composer/formatter.py:14:4: W0107: Unnecessary pass statement (unnecessary-pass)

***** Module smoosh.analyzer.tree

src/smoosh/analyzer/tree.py:3:0: W0611: Unused import os (unused-import)

***** Module smoosh.analyzer.repository

src/smoosh/analyzer/repository.py:35:8: C0415: Import outside toplevel (tree.generate_tree) (import-outside-toplevel)

src/smoosh/analyzer/repository.py:40:0: R0914: Too many local variables (19/15) (too-many-locals)

src/smoosh/analyzer/repository.py:69:8: W1203: Use lazy % formatting in logging functions (logging-fstring-interpolation)

src/smoosh/analyzer/repository.py:72:8: W1203: Use lazy % formatting in logging functions (logging-fstring-interpolation)

src/smoosh/analyzer/repository.py:117:19: W0718: Catching too general exception Exception (broad-exception-caught)

src/smoosh/analyzer/repository.py:118:16: W1203: Use lazy % formatting in logging functions (logging-fstring-interpolation)

src/smoosh/analyzer/repository.py:148:15: W0718: Catching too general

```
exception Exception (broad-exception-caught)

src/smoosh/analyzer/repository.py:149:12: W1203: Use lazy % formatting
in logging functions (logging-fstring-interpolation)

src/smoosh/analyzer/repository.py:156:4: W0107: Unnecessary pass
statement (unnecessary-pass)

src/smoosh/analyzer/repository.py:3:0: W0611: Unused import os (unused-
import)

-----

Your code has been rated at 8.98/10 (previous run: 8.98/10, +0.00)
```

Appendix B — Complete final Pylint output

```
***** Module smoosh.composer.concatenator

src/smoosh/composer/concatenator.py:150:5: W0511: TODO: Implement fold
mode composition (fixme)
```

```
src/smoosh/composer/concatenator.py:166:5: W0511: TODO: Implement  
smoosh mode composition (fixme)
```

Your code has been rated at 9.96/10 (previous run: 9.96/10, +0.00)